SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)
Assessment Tool Weightage: 40%

QUESTIONS:5 **Total Marks:25**

ANNEXURE A
APPLICATION BASED QUESTIONS

Code of Conduct:

I D. Srujana Bai (192571067) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

D. Srujana Bai
Signature of the student:

14.5

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes

of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.

5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)	
Q1	3	4	2	4	2	15
Q2	4	2	4	4	1.5	15.5
Q3	2	4	2	3	1	12
Q4	3	5	4	3	2	17
Q5	3	4	2	4	2	15
						74.5

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	4 Clearly understands the problem and accurately identifies all requirements	3 Good understanding with minor gaps	2 Basic understanding; some requirements unclear	1 Poor understanding or misinterpretation of the problem
Application of Concepts	30	6 Accurately applies appropriate concepts to solve complex problems	5 Applies relevant concepts correctly with minor errors	4 Applies basic concepts with limited accuracy	1-2 Fails to apply appropriate concepts
Problem-Solving Approach	20	4 Logical, well-structured, and efficient approach	3 Mostly logical approach with minor gaps	2 Basic approach with some inconsistencies	1 Illogical or unclear approach
Accuracy of Solution	20	4 Completely correct solution with proper justification	3 Mostly correct with minor errors	2 Partially correct solution	1 Incorrect or incomplete solution
Clarity	10	2 Clear, well-organized, and properly explained solution	1.5 Understandable with minor clarity issues	1 Limited clarity and explanation	0 Poorly presented and difficult to understand

ANSWERS:

Q1. Smart City Surveillance System

1. Problem Identification

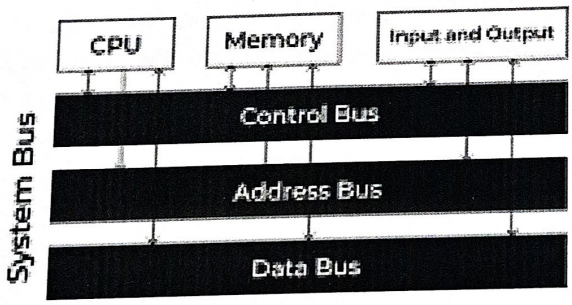
A smart city surveillance system receives video streams from many cameras simultaneously. During peak hours, a large amount of data is transferred between I/O devices, memory, and the CPU, causing delays in threat detection and system response.

2. Objective

To analyze how CPU, memory, and I/O interactions affect performance and to identify ways to improve system efficiency using suitable bus structures and instruction execution techniques.

3. Principle/Concept

The CPU processes video data stored in memory, while cameras act as I/O devices that continuously send information. The bus serves as the communication path between these components. If the bus becomes overloaded, data transfer slows down, reducing overall system performance.



Types of Buses in Computer Architecture

4. Methodology

- Analyze the flow of video data from cameras to memory and CPU.
- Compare single-bus and multi-bus architectures.
- Study how instruction execution cycles influence processing speed.
- Identify bottlenecks causing delays.

5. Tools/Techniques/Algorithms

- Single-Bus Architecture
- Multi-Bus Architecture

- Pipelining
- Direct Memory Access (DMA)
- Cache Memory

6. Data Analysis and Interpretation

In a single-bus system, all components share the same communication path. When many cameras send data simultaneously, the bus becomes congested, increasing waiting time. In a multi-bus system, different buses handle different operations, allowing parallel data transfer and reducing delays. Faster instruction execution and pipelining also improve processing efficiency.

The system can process video feeds more efficiently with reduced lag and faster threat detection. Multi-bus architecture and optimized instruction execution improve overall performance.

8. Application

Used in smart city monitoring, traffic control systems, public safety surveillance, and security management systems.

Q2. Real-Time Stock Trading Application

1. Problem Identification

A stock trading application processes millions of instructions every second. During high trading activity, transaction processing becomes slow because of a high CPI (Cycles Per Instruction) caused by frequent memory access and branch instructions.

2. Objective

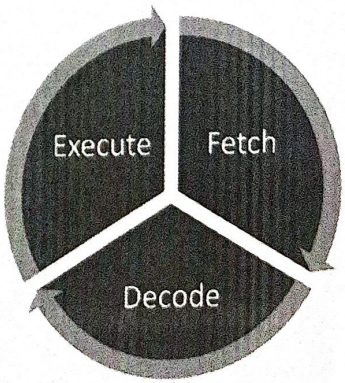
To examine how the fetch-decode-execute cycle affects execution time and identify methods to reduce CPI and improve processor performance.

3. Principle/Concept

Every instruction passes through the fetch, decode, and execute stages. If memory access takes longer or branch instructions cause incorrect predictions, additional clock cycles are required, increasing CPI and execution time.

4. Methodology

- Analyze instruction execution stages.
- Identify causes of high CPI.
- Evaluate memory access patterns and branch behavior.
- Suggest architectural improvements.



5. Tools/Techniques/Algorithms

- Instruction Pipelining
- Branch Prediction
- Cache Memory
- Superscalar Processing
- Out-of-Order Execution

6. Data Analysis and Interpretation

Frequent memory access creates delays because the processor waits for data retrieval. Branch instructions may interrupt the pipeline when prediction errors occur. Cache memory reduces memory access time, while branch prediction minimizes execution interruptions, resulting in lower CPI.

7. Expected Outcome

Reduced transaction processing time, improved CPU utilization, and faster response during heavy trading periods.

8. Application

Used in stock exchanges, financial trading platforms, banking systems, and real-time transaction processing environments.

Q3. 8085-Based Temperature Control Unit

1. Problem Identification

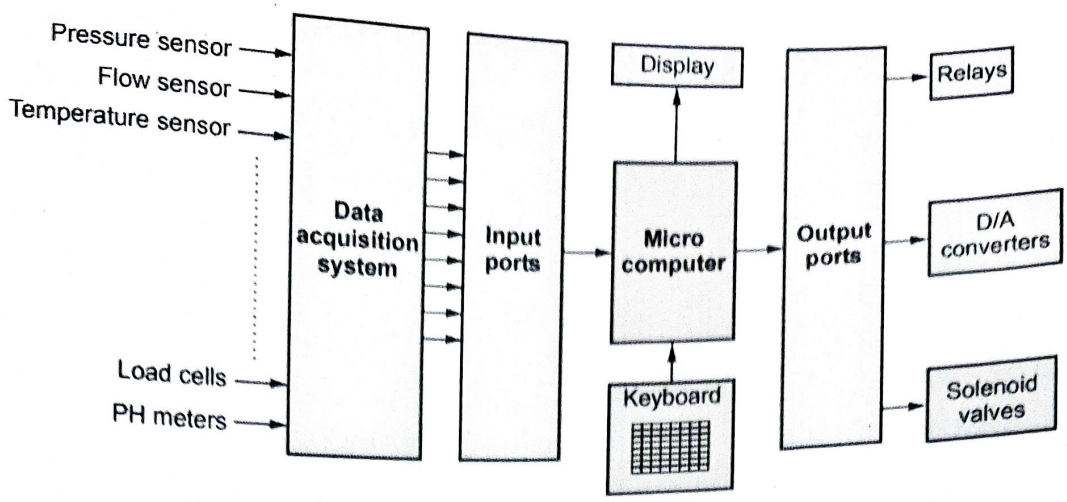
An industrial temperature control system based on the 8085 microprocessor sometimes processes incorrect temperature readings because of improper use of addressing modes in the program.

2. Objective

To understand the impact of addressing modes on data access and improve the accuracy of temperature monitoring and control.

3. Principle/Concept

Addressing modes determine how the processor accesses data. If the wrong addressing mode is selected, the processor may read incorrect memory locations, leading to inaccurate temperature values.



4. Methodology

- Examine different addressing modes used in the program.
- Identify incorrect data access points.
- Use suitable 8085 instructions for sensor data processing.
- Validate data before actuator response.

5. Tools/Techniques/Algorithms

- Immediate Addressing
- Direct Addressing
- Register Addressing
- Indirect Addressing
- Data Validation Techniques

6. Data Analysis and Interpretation

Incorrect memory references may cause the processor to read invalid sensor values. Proper use of direct and indirect addressing ensures that the correct memory locations are accessed. Accurate instruction selection improves system reliability.

7. Expected Outcome

Correct temperature readings, stable system operation, and reduced control errors.

8. Application

Used in industrial automation, HVAC systems, manufacturing plants, and temperature monitoring systems.

Q4. 8086 Segmented Memory System

1. Problem Identification

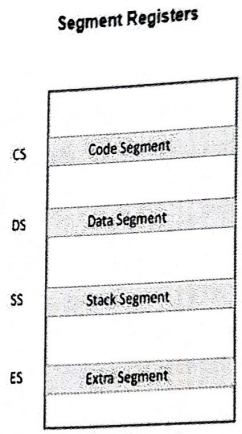
A system using the 8086 microprocessor experiences incorrect data access and stack overflow problems while executing multiple programs.

2. Objective

To analyze the role of segmentation in memory management and ensure proper program execution.

3. Principle/Concept

The 8086 uses segmented memory consisting of Code Segment (CS), Data Segment (DS), and Stack Segment (SS). Each segment stores a specific type of information and is accessed through segment registers.



4. Methodology

- Examine segment allocation.
- Analyze memory access patterns.
- Monitor stack operations.
- Verify addressing mode usage.

5. Tools/Techniques/Algorithms

- Segmentation
- Stack Management
- Segment Registers
- Addressing Modes
- Memory Protection Techniques

6. Data Analysis and Interpretation

Incorrect segment register values can cause the processor to access the wrong memory area. Excessive push operations without corresponding pop operations may result in stack overflow. Proper segment management ensures accurate execution and memory utilization.

7. Expected Outcome

Reliable program execution, correct memory access, and prevention of stack-related errors.

8. Application

Used in multitasking systems, embedded applications, industrial controllers, and legacy computing systems.

Q5. Data Communication System

1. Problem Identification

A software engineer incorrectly interprets hexadecimal packet data, resulting in errors during data processing and communication.

2. Objective

To study the importance of hexadecimal-to-binary conversion and understand its effect on instruction execution and system performance.

3. Principle/Concept

Computers process information in binary form, while hexadecimal notation is commonly used for readability and debugging. Accurate conversion between these number systems is essential for correct data interpretation.

4. Methodology

- Convert hexadecimal values into binary form.
- Verify packet information before processing.
- Analyze instruction execution based on converted data.
- Use benchmarking to evaluate system performance.

5. Tools/Techniques/Algorithms

- Number System Conversion
- Benchmarking
- Error Detection Techniques
- CPU Performance Analysis

6. Data Analysis and Interpretation

An incorrect conversion may change the binary representation of data, leading to wrong instructions or packet values being processed. Efficient CPU design and benchmarking tools help identify such errors quickly and improve system reliability.

7. Expected Outcome

Accurate packet processing, fewer communication errors, and improved system performance.

8. Application

Used in network communication, protocol analysis, embedded systems, cybersecurity, and debugging tools.